Дранцев Александр Павлович
С23-722

Лабораторная работа № 4

Дранцев. С23-722. Вариант 1.

8

Задание 1. Шиномонтаж

N чека	радиус колеса	работа	хранились ли в сто	скидка	цена
A ключевой	B неключевой	C ключевой	D неключевой	E неключевой	F неключевой

$\{A, C\} \to \{B, D, F\}$ + $D \to E$ + $\{B, C\} \to F$ + ~~$\{A, B, C, D\} \to \{E, F\}$~~ ~~$E, F$ – неключевые~~

Возможный полный ключ: $\{A, C\} \to \{B, D, E, F\}$ ~~Возможный полный ключ: $\{A, B, C, D\}$~~ ~~A, B, C, D – ключевые~~

Таблица находится в 1НФ, тк. её значения атомарны (на пересечении столбцов и строк находится одно значение).

Таблица ~~не~~ находится в 2НФ, тк. ~~не все атрибуты E не зависят от ключа~~ все атрибуты полностью зависят от ключа. Не находится в 3НФ тк есть транзитивные связи ~~от полного ключа (привратного)~~. По теореме Хита декомпозируем таблицу на таблицы T_1 и T_2

какая?

T_1: | A | B | C | D | F | + T_2: | D | E |

к ~~нек~~ к ~~нек~~ нек к нек

$\{A, B, C\} \to F$ $D \to E$

$\{A, C\} \to D$ ключи: $\{A, C\}$, $\{B, C\}$ ключ: D

В T_2 значения атомарны → находится в 1НФ. Неключ. значения целиком зависят от ключа => находится в 2НФ. В таблице всего два атрибута => нет транзитивных зависимостей => находится в 3НФ.

$\{A, C\} \to \{B, D, F\}$

$\{B, C\} \to F$

В таблице T_2 один ключ => НФБК совпадает с 3НФ => => находится в НФБК.

В T_1 значения атомарны => находится в ~~НФБК~~ 1НФ. Неключевые ~~зависят~~ знач. целиком зависят от ключа => находится в 2НФ. T_1 не соотв. 3НФ тк. есть транзитивные зависимости. По т-ме Хита декомпозируем таблицу на T_3 и T_4

К-3

T_3: | A | C | D | B | + T_4: | B | C | F |

к к нек нек ~~нек~~ к нек

$\{A, C\} \to \{D, B\}$ $\{B, C\} \to F$

В T_3 значения атомарны => находится в 1НФ. Неключ. атрибуты целиком зависят от ключа => находится в 2НФ. Нет транзитивных зависимостей => находится в 3НФ. Ключ всего один => НФБК совпадает с 3НФ и таблица находится в НФБК.

~~В~~ T_4 полностью аналогично T_3 находится в 1НФ, 2НФ, 3НФ и НФБК.

T_3 – ключ: $\{A, C\}$, T_4 – ключ $\{B, C\}$, T_2 – ключ D [redacted]

4

Итог: T_3 [A|C|D|B] T_4 [B|C|F] T_2 [D|E]. Ключ T_3: {A,C}, T_4: {B,C}, T_2: D.
к к нек нек к к нек к нек

№2. A – номер аудитории
B – кол-во мест
C – есть ли проектор
D – можно ли проводить лекции
E – корпус
F – основная площадка или нет.

$E \to F$ +
$B \to D$ +
$\{A,E\} \to \{B,C,D,F\}$ +

Возможный полный ключ: $\{A,E\} \to \{B,C,D,F\}$. Таблица находится в 1НФ тк. все значения атомарны. Таблица находится в ~~2НФ, тк~~ все неключевые атрибуты зависят от ключа целиком. ~~Не находится в 3НФ тк.~~ не правильно! есть транзитивные связи. По т-ме Хита декомпозируем на T_1 и T_2

T_1 [A|E|B|C|D] + [E|F] T_2 + T_2 находится в 1НФ
к к нек нек нек к нек
тк. значения атомарны,
$\{A,E\} \to \{B,C,D\}$ $E \to F$ находится в 2НФ тк неключевые
$\{B \to D\}$
эл. целиком зависят от ключа. В таблице 2 атрибута => нет транзитивных зависимостей => находится в 3НФ. В таблице всего 1 ключ => НФБК совпадает с 3НФ => табл. находится в НФБК.

T_1 находится в 1НФ тк значения атомарны, находится в 2НФ тк неключевые ~~зависимости~~ значения целиком зависят от ключа. Не находится в 3НФ тк есть транзитивные зависимости. Декомпозируем по т-ме Хита на T_3 и T_4.

T_3 [A|E|B|C] [B|D] T_4.
к к нек нек к нек

$\{A,E\} \to \{B,C\}$ $B \to D$

4

T_4 находится в 1НФ тк значения атомарны, находится в 2НФ тк неключевые атрибуты зависят целиком от ключа. В таблице 2 атрибута => нет транзитивных связей => находится в 3НФ. В таблице 1 ключ => НФБК совпадает с 3НФ => находится в НФБК.

Лабораторная работа № 5

Задание 0.

Триггер не позволяет назначать «бесплатным» больным лечение дороже 5000 руб.

Решение

```
CREATE OR REPLACE FUNCTION trg_check_free_patient_cost()
  RETURNS trigger AS
$$
DECLARE
  paid_flag boolean;
BEGIN

  SELECT pc.status_pay
    INTO paid_flag
  FROM public.period_ill pi
  JOIN public.patients_card pc ON pi.patient_id = pc.patient_id
  WHERE pi.period_id = NEW.period_id;

  IF paid_flag = FALSE AND NEW.cost > 5000 THEN
    RAISE EXCEPTION 'Бесплатный пациент cost % > 5000 запрещено',
      NEW.cost;
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER check_free_patient_cost
  BEFORE INSERT OR UPDATE
  ON public.medical_results
  FOR EACH ROW
  EXECUTE FUNCTION trg_check_free_patient_cost();
```

Задание 1.

Позволяет по номеру карты пациента вычислить суммарную стоимость оказанных ему услуг (действий).

Решение

```
CREATE OR REPLACE FUNCTION total_cost_by_patient(p_patient_id integer)
  RETURNS numeric(10,2) AS
$$
DECLARE
  sum_cost numeric(10,2);
BEGIN
  SELECT COALESCE(SUM(mr.cost),0)
    INTO sum_cost
  FROM public.period_ill pi
  JOIN public.medical_results mr ON pi.period_id = mr.period_id
  WHERE pi.patient_id = p_patient_id;
```

```
  RETURN sum_cost;
END;
$$ LANGUAGE plpgsql;
```

Задание 2.

Для множества пациентов (номера карт), определить суммарное количество выполненных действий (обследований и т.д.).

Решение

```
CREATE OR REPLACE FUNCTION agg_count_actions(state integer, p_patient_id integer)
  RETURNS integer AS
$$
DECLARE
  cnt integer;
BEGIN
  SELECT COUNT(*)
    INTO cnt
  FROM public.period_ill pi
  JOIN public.medical_results mr ON pi.period_id = mr.period_id
  WHERE pi.patient_id = p_patient_id;
  RETURN state + cnt;
END;
$$ LANGUAGE plpgsql;

DROP AGGREGATE IF EXISTS total_actions(integer);
CREATE AGGREGATE total_actions(integer) (
  SFUNC    = agg_count_actions,
  STYPE    = integer,
  INITCOND = '0'
);
```

Задание 3.

Создайте представление, отображающее ФИО больного, дату его поступления (начала периода) и номер палаты (и ключевые поля таблицы, описывающей периоды пребывание больного), где он лежит. Реализуйте возможность изменения номера палаты через это представление в реальной таблице.

Решение

```
CREATE OR REPLACE VIEW patient_stay AS
SELECT
  pi.period_id,
  pc.patient_id,
  pc.last_name || ' ' || pc.first_name || ' ' || pc.second_name AS full_name,
  pi.period_start,
  pi.room_id
FROM public.patients_card pc
JOIN public.period_ill pi USING(patient_id);
```

```
CREATE OR REPLACE FUNCTION trg_patient_stay_update()
  RETURNS trigger AS
$$
BEGIN
  UPDATE public.period_ill
    SET room_id = NEW.room_id
  WHERE period_id = NEW.period_id;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
DROP TRIGGER IF EXISTS patient_stay_instead_of_update ON patient_stay;

CREATE TRIGGER patient_stay_instead_of_update
  INSTEAD OF UPDATE ON patient_stay
  FOR EACH ROW EXECUTE FUNCTION trg_patient_stay_update();
```

Задание 4

При помощи таблицы и набора процедур (или функций) реализуйте структуру представления данных «Стек». Структура должна позволять сохранять в стек строки, длиной не более 64 символов. Должны быть доступны действия push – добавление элемента на вершину стека, pop – удаление элемента с вершины стека, empty – очистка стека, init – инициализация стека, top – просмотр вершины стека.

Решение

```
CREATE TABLE IF NOT EXISTS stack_data (
  stack_name text    NOT NULL,
  position   integer NOT NULL,
  value      varchar(64) NOT NULL,
  PRIMARY KEY (stack_name, position)
);

CREATE OR REPLACE FUNCTION stack_init(p_stack text) RETURNS void AS
$$
BEGIN
  DELETE FROM stack_data
   WHERE stack_name = p_stack;
END;
$$ LANGUAGE plpgsql;
CREATE OR REPLACE FUNCTION stack_push(p_stack text, p_val text) RETURNS void AS
$$
DECLARE
  maxpos integer;
BEGIN
  SELECT COALESCE(MAX(position),0) INTO maxpos
    FROM stack_data
   WHERE stack_name = p_stack;

  INSERT INTO stack_data(stack_name, position, value)
    VALUES(p_stack, maxpos+1, LEFT(p_val,64));
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION stack_pop(p_stack text) RETURNS varchar(64) AS
$$
DECLARE
  res    varchar(64);
  maxpos integer;
BEGIN
  SELECT MAX(position) INTO maxpos
    FROM stack_data
   WHERE stack_name = p_stack;

  IF maxpos IS NULL THEN
    RETURN NULL;
  END IF;

  SELECT value INTO res
    FROM stack_data
   WHERE stack_name = p_stack
     AND position = maxpos;

  DELETE FROM stack_data
   WHERE stack_name = p_stack
     AND position   = maxpos;

  RETURN res;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION stack_top(p_stack text) RETURNS varchar(64) AS
$$
DECLARE
  res    varchar(64);
  maxpos integer;
BEGIN
  SELECT MAX(position) INTO maxpos
    FROM stack_data
   WHERE stack_name = p_stack;

  IF maxpos IS NULL THEN
    RETURN NULL;
  END IF;

  SELECT value INTO res
    FROM stack_data
   WHERE stack_name = p_stack
     AND position   = maxpos;

  RETURN res;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION stack_empty(p_stack text) RETURNS void AS
$$
BEGIN
  DELETE FROM stack_data
```

```
  WHERE stack_name = p_stack;
END;
$$ LANGUAGE plpgsql;
```

Задание на защиту:

Чтобы узнавалось на какое время палата была занята.

Решение:

```
DROP TRIGGER IF EXISTS patient_stay_instead_of_update ON patient_stay;
DROP FUNCTION IF EXISTS trg_patient_stay_update();

CREATE OR REPLACE FUNCTION trg_patient_stay_update()
 RETURNS trigger
AS
$$
DECLARE
 existing_start DATE;
 existing_end   DATE;
 cnt            INTEGER;
BEGIN
 IF NEW.room_id <= 0 THEN
  RAISE EXCEPTION 'Неправильный room_id %: надо положительный', NEW.room_id;
 END IF;

 SELECT period_start, period_end
  INTO existing_start, existing_end
 FROM public.period_ill
 WHERE period_id = NEW.period_id;

 SELECT COUNT(*)
  INTO cnt
 FROM public.period_ill AS pi
 WHERE pi.room_id = NEW.room_id
  AND pi.period_id <> NEW.period_id
  AND pi.period_start <= COALESCE(existing_end, CURRENT_DATE)
  AND COALESCE(pi.period_end, CURRENT_DATE) >= existing_start;

 IF cnt > 0 THEN
  RAISE EXCEPTION 'Комната % уже занята между % и %',
   NEW.room_id, existing_start, existing_end;
 END IF;

 UPDATE public.period_ill
  SET room_id = NEW.room_id
 WHERE period_id = NEW.period_id;

 RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER patient_stay_instead_of_update
 INSTEAD OF UPDATE ON patient_stay
 FOR EACH ROW
 EXECUTE FUNCTION trg_patient_stay_update();
```

Лабораторная работа № 6

Задание 0.

Запрос, выбирающий без повторений номера палат, номера отделений, число пациентов, лежащих в настоящее время в палате, число пациентов, лечащих в настоящее время в отделении.

Решение

```
SELECT DISTINCT
  hr.number_room,
  hr.department_id,
  COUNT(pi.patient_id)
    OVER (PARTITION BY hr.room_id)  AS patients_in_room,
  COUNT(pi.patient_id)
    OVER (PARTITION BY hr.department_id) AS patients_in_department
FROM public.hospital_room AS hr
LEFT JOIN public.period_ill AS pi
  ON pi.room_id = hr.room_id
    AND pi.period_start <= CURRENT_DATE
    AND (pi.period_end   >= CURRENT_DATE OR pi.period_end IS NULL)
ORDER BY hr.number_room;
```

Задание 1.

Запрос, выбирающий номер карты пациента и в виде одного столбца через точку с запятой все его обследования в порядке их времени проведения.

Решение

```
SELECT
 pc.patient_id,
  string_agg(mr.name_result, '; ' ORDER BY mr.date_start) AS all_examinations
FROM public.patients_card    AS pc
JOIN public.period_ill      AS pi ON pc.patient_id = pi.patient_id
JOIN public.medical_results AS mr ON pi.period_id  = mr.period_id
GROUP BY pc.patient_id
ORDER BY pc.patient_id;
```

Лабораторная работа № 7

```
from dbtable import *

class RoomTable(DbTable):
    def table_name(self):
        return self.dbconn.prefix + "rooms"

    def columns(self):
        return {"id": ["varchar(32)", "NOT NULL", "PRIMARY KEY"]}

    def find_by_position(self, num):
```

```
        sql = "SELECT id FROM " + self.table_name()
        sql += " ORDER BY "
        sql += ", ".join(self.primary_key())
        sql += " LIMIT 1 OFFSET %(offset)s"
        cur = self.dbconn.conn.cursor()
        cur.execute(sql, {"offset": num - 1})
        return cur.fetchone()

from dbtable import *

class DepartmentTable(DbTable):
    def table_name(self):
        return self.dbconn.prefix + "departments2"

    def columns(self):
        return {"department": ["varchar(12)", "NOT NULL"],"room_id": ["varchar(32)", "REFERENCES rooms(id)"]}

    def primary_key(self):
        return ['room_id', 'department']

    def table_constraints(self):
        return ["PRIMARY KEY(room_id, department)"]

    def all_by_room_id(self, pid):
        s = str(pid)
        sql = "SELECT * FROM " + self.table_name()
        sql += " WHERE room_id = '%s'" % s
        sql += " ORDER BY "
        sql += ", ".join(self.primary_key())
        cur = self.dbconn.conn.cursor()
        cur.execute(sql)
        return cur.fetchall()
```

```
Добро пожаловать!
Основное меню (выберите цифру в соответствии с необходимым действием):
    1 - просмотр палат;
    2 - сброс и инициализация таблиц;
    9 - выход.
=> 2
Таблицы созданы заново!
Добро пожаловать!
Основное меню (выберите цифру в соответствии с необходимым действием):
    1 - просмотр палат;
    2 - сброс и инициализация таблиц;
    9 - выход.
=> 1
Просмотр списка палат!
№ палаты
Test1
Test2
Test3
Дальнейшие операции:
    0 - возврат в главное меню;
    3 - добавление новой палаты;
    4 - удаление палаты;
    5 - просмотр отделения палаты;
    9 - выход.
```